NFA TO DFA CONVERSION SIMULATOR

Comprehensive System Specification, Architecture, Algorithmic Analysis & Validation Report

Target Course: Automata Theory / Theory of Computation	**Author:** Kabir Nagdev
Technology Stack: Python 3.13, Streamlit, Graphviz, Pytest	**Repository:** github.com/kabirnagdev/NFA-to-DFA-converter

1. Existing-Tool Analysis & Comparative Evaluation

Undergraduate computer science instruction in Formal Languages and Automata Theory has traditionally relied on desktop utilities or basic client-side scripts. To establish design goals, four representative tools were evaluated:

- **JFLAP 7.1 (Duke University)**: Extensive theoretical scope across Turing machines, grammars, and PDAs. However, it requires a local Java JRE runtime, suffers from dated multi-window modal dialogs, and exhibits UI scaling flaws on high-DPI displays.
- **Cyberzhg Automata Simulator (cyberzhg.github.io)**: Provides instant browser-based execution via Vis.js, but operates as a *black box*—converting NFAs directly into DFAs without revealing the step-by-step subset unions, omitting explicit dead state self-loops, and lacking dual synchronous traces.
- **Automata-lib / FSMView (Python Package Ecosystem)**: High algorithmic correctness, but lacks any graphical user interface, making it unsuited for classroom interaction or non-programmer evaluation.
- **Stanford Automata Applets**: Simple interactive canvas, but unmaintained, restricted to fixed alphabet sizes, and lacking automated testing.

Table 1.1: Feature Comparison Matrix

Dimension	JFLAP 7.1	Cyberzhg Web	automata-lib	Our Simulator
Delivery	Java App (.jar)	Client JS / Web	Python CLI / Lib	**Streamlit Web App**
Input Interface	Drag-and-drop nodes	Raw text / Canvas	Python code / Dict	**Interactive Data Grid**
Derivation Trace	Step dialogs	Black-box output	None (Programmatic)	**Granular BFS Union Trace**
Trap State (∅)	Implicit	Often omitted	Standard	**Explicit ∅ Self-Loops**
String Simulation	Single trace stepper	Final boolean only	Programmatic eval	**Dual Synchronous Trace**
Automated Tests	None	None	Pytest available	**18 Pytest Tests (100%)**

2. System Architecture & UML Specifications

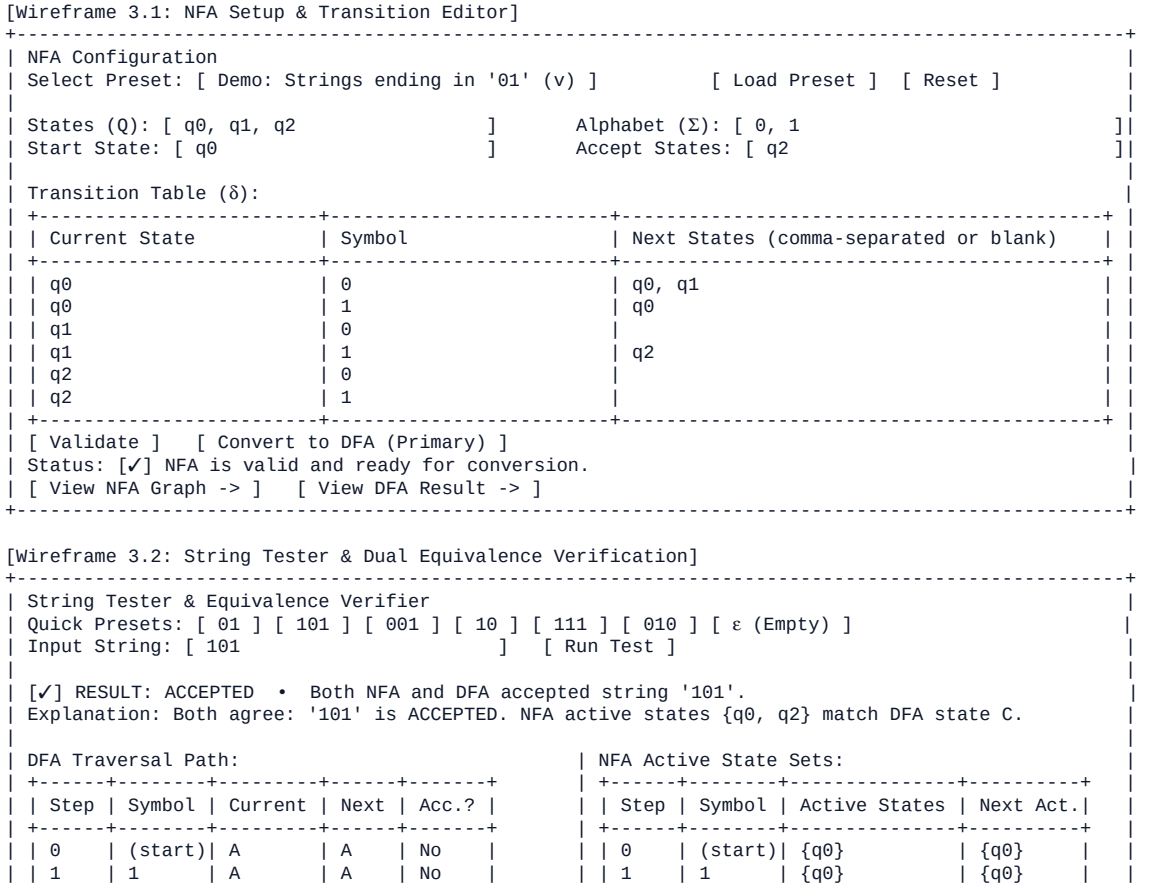
The simulator employs a decoupled, modular 3-tier architecture separating presentation, controller state, and automata domain logic:

- **Presentation Layer**: Built using Streamlit, comprising 6 dedicated pages (Overview, NFA Setup, NFA Graph, DFA Result, Step Trace, String Tester). Custom CSS provides crisp academic styling.
- **Controller & State Layer**: Manages multi-step session persistence and safe deferred navigation (nav_target) to prevent widget instantiation conflicts across reruns.
- **Core Domain & Algorithmic Engine**: Contains NFA, DFA, SubsetConstructionEngine, NFAValidator, and AutomataSimulator.
- **Visualization Layer**: Generates standard Graphviz DOT definitions rendering initial start arrows, double circles for final states, and consolidated transition labels.

2.1 Class Model & Component Interactions

3. User Interface Mock-Ups & Layout Specifications

The user interface emphasizes clean information hierarchy, readable typography, and accessible controls. Below are the structural wireframes:



```
| | 2    | 0      | A       | B    | No    |       | | 2    | 0      | {q0}          | {q0, q1} |   |
| | 3    | 1      | B       | C    | Yes(*)|       | | 3    | 1      | {q0, q1}      | {q0, q2}*|   |
| +------+--------+---------+------+-------+       | +------+--------+---------------+----------+   |
+-------------------------------------------------------------------------------------------------------+
```

4. Formal Algorithm Specification & Complexity

Definition (Rabin-Scott Powersets): Given an NFA M = (Q, Σ, δ, q0, F), the equivalent DFA M' = (Q', Σ, δ', q'0, F') is constructed such that q'0 = {q0}, δ'(S, a) = ■_{q ∈ S} δ(q, a), and F' = {S ∈ Q' | S ∩ F ≠ ∅}. The BFS algorithm below explores reachable states only:

```
Algorithm: SUBSET_CONSTRUCTION
Input    : NFA M = (Q, Σ, δ, q0, F)
Output   : ConversionResult(DFA M', steps, state_mapping)

1:  Let queue be an empty FIFO queue (collections.deque)
2:  Let subset_to_label be an empty hash map (frozenset -> str)
3:  Let label_to_subset be an empty hash map (str -> frozenset)
4:  Let dfa_transitions be an empty hash map ((str, str) -> str)
5:  Let steps be an empty list of ConversionStep records
6:  Let label_counter ← 0
7:
8:  // Step 1: Initialize Start State
9:  S0 ← frozenset({q0})
10: initial_label ← GENERATE_LABEL(label_counter)  // 'A'
11: label_counter ← label_counter + 1
12: subset_to_label[S0] ← initial_label
13: label_to_subset[initial_label] ← S0
14: queue.ENQUEUE(S0)
15:
16: // Step 2: BFS Exploration of Reachable Subsets
17: while queue is not empty do
18:     current_subset ← queue.DEQUEUE()
19:     current_label ← subset_to_label[current_subset]
20:
21:     for each symbol a ∈ SORTED(Σ) do
22:         target_subset ← ∅
23:         individual_trans ← empty dictionary
24:         if current_subset == ∅ then
25:             target_subset ← frozenset()   // Trap state loops to itself
26:         else
27:             for each state q ∈ SORTED(current_subset) do
28:                 targets ← δ(q, a)
29:                 individual_trans[q] ← targets
30:                 target_subset ← target_subset ∪ targets
31:             end for
32:             target_subset ← frozenset(target_subset)
33:         end if
34:
35:         is_new ← FALSE
36:         if target_subset ∉ subset_to_label then
37:             target_label ← GENERATE_LABEL(label_counter)
38:             label_counter ← label_counter + 1
39:             subset_to_label[target_subset] ← target_label
40:             label_to_subset[target_label] ← target_subset
41:             queue.ENQUEUE(target_subset)
42:             is_new ← TRUE
43:         else
44:             target_label ← subset_to_label[target_subset]
45:         end if
46:         dfa_transitions[(current_label, a)] ← target_label
47:
48:         // Record step trace for pedagogical breakdown
49:         steps.APPEND(ConversionStep(step=LENGTH(steps)+1, current_label, current_subset,
50:                                     a, individual_trans, target_subset, target_label, is_new))
51:     end for
52: end while
53:
54: // Step 3: Classify Accepting States
55: dfa_final_states ← { L for L, S in label_to_subset.items() if S ∩ F ≠ ∅ }
56: return DFA(states=KEYS(label_to_subset), Σ, dfa_transitions, initial_label, dfa_final_states)
```

Complexity Bounds: Worst-case time is O(2^|Q| · |Σ| · |Q|), but BFS explores only reachable subsets |Q'| ■ 2^|Q| (e.g. for |Q|=10, 2^10 = 1024, but typical practical recognizers have |Q'| ≤ 20). String simulation runs in linear O(|w|) time on the DFA and O(|w| · |Q|) on the NFA.

5. Manually Solved Validation Example & Trace Verification

Problem Specification: Convert the NFA accepting all binary strings ending with '01' into an equivalent DFA, and verify acceptance on test strings 101, 110, and ε.

States Q: {q0, q1, q2} | **Alphabet** Σ: {0, 1} | **Start**: q0 | **Final F**: {q2}

NFA Transitions δ: δ(q0, 0) = {q0, q1}; δ(q0, 1) = {q0}; δ(q1, 1) = {q2}; others = ∅

5.1 Step-by-Step Manual Derivation

Step 0 (Initialization): Initial state S0 = {q0} ■ Label **A** = {q0}. Queue = [A].

Iteration 1 (Process A = {q0}):
- On symbol 0: δ'(A, 0) = δ(q0, 0) = {q0, q1}. New subset ■ Label **B** = {q0, q1}. Enqueue B.
- On symbol 1: δ'(A, 1) = δ(q0, 1) = {q0}. Existing subset ■ State **A**.

Iteration 2 (Process B = {q0, q1}):
- On symbol 0: δ'(B, 0) = δ(q0, 0) ∪ δ(q1, 0) = {q0, q1} ∪ ∅ = {q0, q1}. Existing subset ■ State **B**.
- On symbol 1: δ'(B, 1) = δ(q0, 1) ∪ δ(q1, 1) = {q0} ∪ {q2} = {q0, q2}. New subset ■ Label **C** = {q0, q2}. Enqueue C.

Iteration 3 (Process C = {q0, q2}):
- On symbol 0: δ'(C, 0) = δ(q0, 0) ∪ δ(q2, 0) = {q0, q1} ∪ ∅ = {q0, q1}. Existing subset ■ State **B**.
- On symbol 1: δ'(C, 1) = δ(q0, 1) ∪ δ(q2, 1) = {q0} ∪ ∅ = {q0}. Existing subset ■ State **A**.
- Queue is empty. BFS terminates.

5.2 Final Accepting States Classification

An equivalent DFA state is accepting if its constituent subset intersects with NFA final states F = {q2}:
- State A = {q0} ■ {q0} ∩ {q2} = ∅ ■ Non-accepting
- State B = {q0, q1} ■ {q0, q1} ∩ {q2} = ∅ ■ Non-accepting
- State C = {q0, q2} ■ {q0, q2} ∩ {q2} = {q2} ≠ ∅ ■ **Accepting (*)**

DFA Final States: F' = {C}

Table 5.1: Complete Converted DFA Transition Matrix

DFA State	Constituent NFA Subset	Input '0'	Input '1'	Accepting Status
→ **A**	{q0}	B ({q0, q1})	A ({q0})	Non-accepting (Start)
B	{q0, q1}	B ({q0, q1})	C ({q0, q2})	Non-accepting
C*	{q0, q2}	B ({q0, q1})	A ({q0})	**Accepting (*)**

5.3 String Execution Verification Traces

String 1: w = 101 (Should be ACCEPTED):
- *NFA Active States*: {q0} ■■1■■> {q0} ■■0■■> {q0, q1} ■■1■■> {q0, q2}*. Final active set contains q2 ∈ F ■ **ACCEPTED**.
- *DFA Traversal*: A ■■1■■> A ■■0■■> B ■■1■■> C*. State C ∈ F' ■ **ACCEPTED**.
- **Result**: Both accept. Equivalence verified.

String 2: w = 110 (Should be REJECTED):
- *NFA Active States*: {q0} ■■1■■> {q0} ■■1■■> {q0} ■■0■■> {q0, q1}. No final state in {q0, q1} ■ **REJECTED**.
- *DFA Traversal*: A ■■1■■> A ■■1■■> A ■■0■■> B. State B ∉ F' ■ **REJECTED**.
- **Result**: Both reject. Equivalence verified.

String 3: w = ε (Empty string ■ Should be REJECTED):
- *NFA* halts in {q0} ∉ F ■ **REJECTED**. *DFA* halts in A ∉ F' ■ **REJECTED**.